VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF ELECTRONICS ENGINEERING

May, 2026

A project report on

DESIGN AND VERIFICATION OF HBM4 - MEMORY ARCHITECTURE

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Electronics and Communication Engineering

by

Y S ADITYA LOHIT

(22BEC1469)

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

DECLARATION

I, Y S ADITYA LOHIT (Registration No: 22BEC1469), hereby declare that the project report titled “Design and Verification of HBM4- Memory Architecture” submitted in partial fulfillment of the requirements for the award of the degree Bachelor of Technology in Electronics and Communication Engineering is the result of my original work carried out under the supervision of my project guide.

I further declare that:

I. This report has not been submitted, wholly or in part, for the award of any other degree or diploma.
II. All sources of information used in this project have been duly acknowledged.
III. The work presented here is based on my independent effort, and any external assistance received has been clearly mentioned in the appropriate sections of this report.
IV. The results, experimentation, implementation, and analysis presented reflect my own findings and contributions.

Place: Chennai

Date: 25.05.2026

Y S ADITYA LOHIT

Signature of the Candidate

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Electronics Engineering

CERTIFICATE

Building greater futures through innovation and collective knowledge

TCS Commitment

In it for good.

Bring everything.

Know-how.

Master the journey.

TATA CONSULTANCY SERVICES

TATA

Internship Certificate

Y S Aditya Lohit

Course: B Tech in ECE

Institute: VIT - Chennai

From **09-Feb-2026** to **30-Apr-2026**

Mentor Name: **Venkateswarlu Unnam**

Project: **HBM4 Verification Using UVM Methodology**

Dr K M Suceendran

Head – Academic Alliances Group

ABSTRACT

The rapid increase in the use of data-intense applications such as AI accelerators, HPC computing systems, and GPU architectures has increased the demand for memory systems with high bandwidths with very little energy consumption. To cater to these needs, HBM4 technology uses a DRAM-based 3D stack that makes use of TSVs and microbumps in connecting different DRAM dies vertically with a Logic Die placed at the bottom and acts as a controller for the host system.

This paper describes an HBM4 Logic Die interface testbench based on UVM (Universal Verification Methodology). The design under test (DUT) is implemented using RTL coding in SystemVerilog and includes features such as 16 bits data bus size, 3 bits address width, 2 banks, and timing values such as tRCD, tCL, and tRP equal to 2 clocks. ACTIVATE, READ, WRITE, and PRECHARGE commands along with the state machine transition and PHY wrapper functionality have been implemented and tested.

The testbench of UVM consisted of layers such as hbm4_item, which represented the transaction class, hbm4_seq_lib which was a collection of sequences, a driver that was compliant with the protocol, a monitor to observe the DUT, the agent to drive the sequence to the bus, a scoreboard to compare the expected and the actual values, and the functional coverage block.

The Static Timing Analysis (STA) showed the timing closure with zero TNS and worst case setup slack of 4.443ns. The functional coverage attained around 94.32%, where all the operations were covered to 100% using the different cover groups like Operation_type, Bank, Row, Column, Data_mask, and Command. All the seven concurrently written SVA assertions were passed successfully.

Keywords: HBM4, UVM, SystemVerilog, Functional Verification, RTL Design, Constrained Random Testing, Coverage Driven Verification, State Machine, STA, Logic Die.

ACKNOWLEDGEMENT

With immense pleasure and a deep sense of gratitude, I wish to express my sincere thanks to the faculty and staff members of the School of Electronics Engineering (SENSE), Vellore Institute of Technology (VIT), Chennai. Their continued support, academic guidance, and encouragement have played a vital role throughout the duration of my work on this dissertation.

I am grateful to the Honorable Chancellor of VIT, **Dr. G. Viswanathan**, the Vice President, and the Vice Chancellor for fostering an inspiring academic and research-oriented environment at the Institute. Their vision and leadership have continually motivated students like me to pursue innovative research.

Finally, I would like to thank **Vellore Institute of Technology** for providing excellent infrastructure, academic resources, and a flexible learning ecosystem that supported every aspect of my project work and research activities related to this dissertation.

Place: Chennai

Date: 25.05.2026 **Y S ADITYA LOHIT**

CONTENTS

LIST OF FIGURES

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

Due to the growing popularity of artificial intelligence (AI), high-performance computing (HPC), data center usage, graphics processing, and memory-intensive computation applications, standard memory structures have hit their peak performance. Conventional memory structures are being limited due to the restrictions in the bandwidth, latency, power efficiency, and scalability.High Bandwidth Memory (HBM) is the answer to these challenges in memory structure by stacking multiple DRAM chips vertically through the use of TSVs and connecting them through a wide interface with a logic die.

In this generation of HBM, HBM4 offers even better performance in terms of memory bandwidth capacity, channel configuration, and overall system-level integration. However, architectural complexity presents difficulties in various aspects like digital verification, protocol compatibility, static timing analysis, and functional correctness.

With the help of verification methodologies like the universal verification methodology (UVM) and static timing analysis (STA), the reliability of these memory structures can be assured. This project aims to create a simple architecture for HBM4 logic die design through RTL and verify its functionality through UVM-based verification techniques.

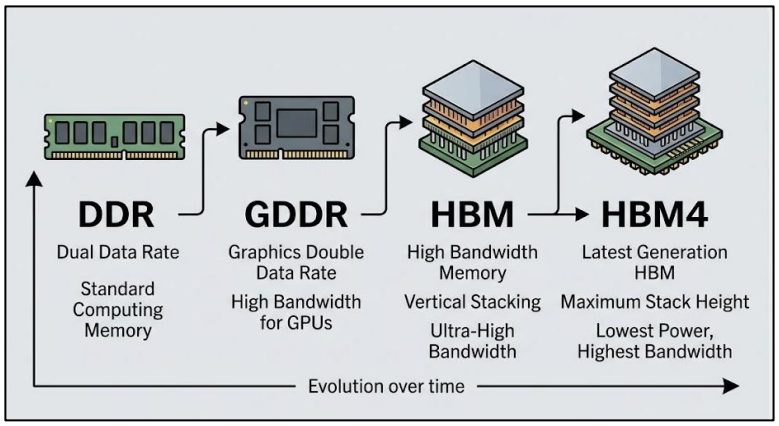


Figure 1 Evolution of memory

1.2 PROBLEM DEFINITION

Modern memory systems have the need for very high bandwidth while keeping latencies low and power dissipation low as well. With the advent of stacked memory architectures like HBM4, the intricacies of dealing with interface, timing, and protocol issues become very significant. Verification techniques using only directed testing would be unable to meet the challenge, since they lack coverage and do not include corner cases.

Moreover, meeting timing constraints becomes another problem to solve as any small timing violation can result in data corruption. Thus, a scalable RTL implementation accompanied by a sophisticated verification environment would be required to meet these challenges.

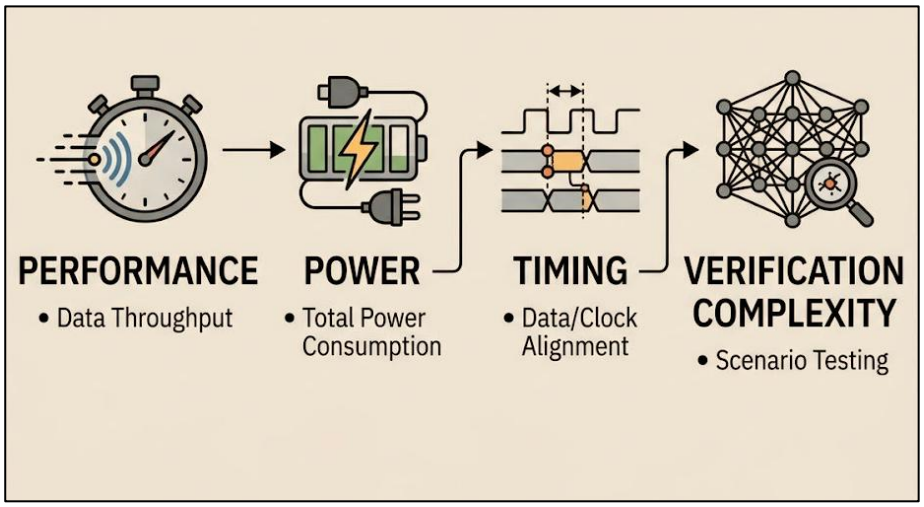


Figure 2 Challenges in High-Speed Memory Verification

1.3 OBJECTIVES OF THIS PROJECT

The major aim of this project is designing, implementation, and comprehensive testing of logic die functionality for upcoming HBM4 architecture. The following technical objectives have been set in order to realize these research aims:

Comprehensive Architecture and Protocol Analysis: Conducting thorough analysis of HBM4 architecture with respect to changes in architecture (such as usage of finer process

technology for fabrication of logic die), signaling interfaces, command protocol, and channel routing complexity in the case of massive parallel channel usage.

Design and RTL Implementation: Modeling and implementation of core logic of control and command decoding, as well as routing of interface using SystemVerilog/RTL with an emphasis on maximum efficiency, high throughput, and protocol compliance.

VIP Architecture and UVM-based Verification: Designing and deployment of comprehensive, highly scalable and reusable verification IP based on UVM methodology which will include various elements, such as agents, drivers, monitors, scoreboards, and sequencers.

Generation of Constrained-Random Test: For building advanced sequence libraries that can generate constrained-random tests to validate the high-speed interface design over an enormous state space, testing its boundary conditions, consecutive accesses, and edge case protocols.

Integration of SystemVerilog Assertions (SVA): For adding a rich system of concurrent and inline SystemVerilog Assertions within the design and interface components to verify the protocol in real time, facilitate quick debugging, and prepare for formal verification.

Static Timing Analysis (STA) and Timing Closures: For carrying out static timing analysis of the design to fix setup/hold violations, issues related to clock skewing, and critical paths to ensure the design meets tight timing requirements.

Analysis of Coverage-Driven Verification (CDV): For defining and analyzing complete coverage requirements, including functional and code coverage (covergroups, coverpoints, and cross-coverage) for verifying the design as per the verification plan.

Simulation and Design Validation: For running comprehensive test regressions, analyzing the waveforms generated by simulations, and measuring performance metrics to prove correctness and reliability of the design.

1.4 SCOPE OF THE PROJECT

This research is precisely confined to the functional design, implementation, and rigorous verification of selected sub-modules of an HBM4 logic die. Considering the high degree of architectural complexity and proprietary aspects of commercial memory designs, the current study provides the foundation for implementing a concrete execution framework for validating the effectiveness of key digital design approaches and verification techniques.

1.4.1 In-Scope Technical Components

The ongoing engineering and verification activities for this project involve the following technical areas:

RTL Architecture & Logic Synthesis: Creation of the basic architecture of the HBM4 logic die based on SystemVerilog. This entails the synthesis of control paths, command registers, and fast interface logics of the chip.

Interface and Memory Controller Modules: Designing of selected interfaces for memory controller logic, such as command decoders, multiplexer networks, and pseudo-independent bank controls that dictate the flow of row/column accesses.

Universal Verification Methodology (UVM) Verification Environment: Creation of a fully layered verification environment based on UVM from scratch. This includes the design of custom sequence generators, driver, monitor, and scoreboard.

Compliance With Protocol and Checker Use: Use of complete protocol checkers and a specific SVA environment, concentrated mainly on validating complicated HBM4 timing constraints (t{RCD}, t{RP}, and t{RAS}) through functional simulation.

Coverage-Based Metrics: Development of an elaborate functional coverage strategy via the use of covergroups, coverpoints, and cross-coverage constructs to express and compute code coverage and state space exploration mathematically.

Pre-Layout Timing Verification: Conducting STA at the gate-level/netlist stage to uncover critical paths, verify the set-up/hold margins, and validate proper clock domain hierarchy suitable for the desired operation frequency.

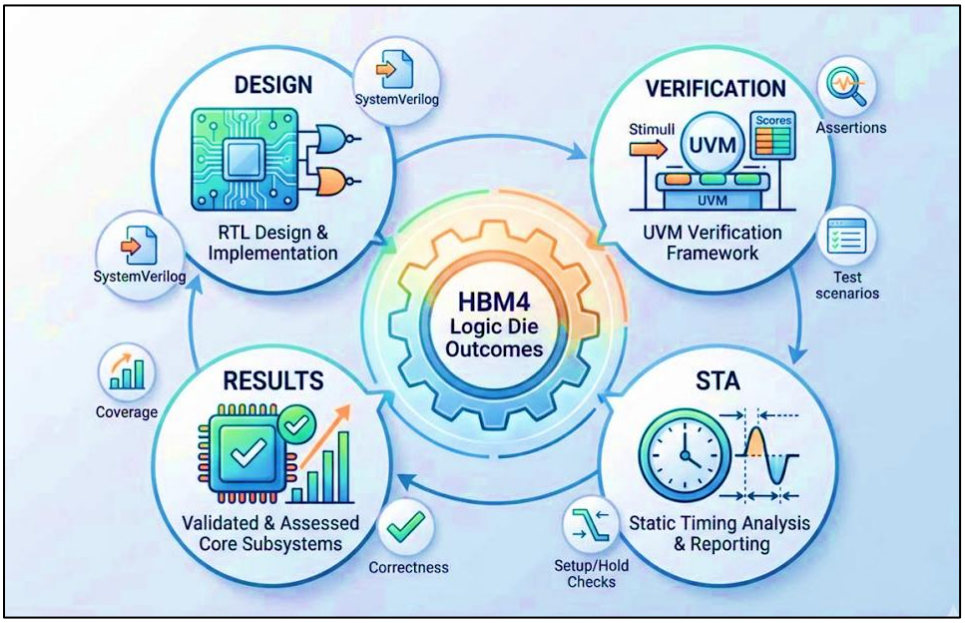


Figure 3 Project Scope Diagram

1.4.2 Components Not Within Scope

For academic reasons, the following domains fall outside the scope of the work at hand:

Complete JEDEC Compliance: Full JEDEC compliance with the HBM4 specification, which would involve testing all commands specified by JEDEC, including initialization sequences, testing modes, and ECC code recovery routines.

Physical Design and Back-end: All back-end tasks related to physical synthesis, including floor planning, CTS, P&R, and RC delay estimation.

Physical Fabrication and Testing: Physical chip layout, fabrication in the foundry, and post-layout validation using oscilloscopes or logic analyzers.

1.5 PROPOSED SOLUTION

The The suggested solution uses HBM4-based memory interface architecture implemented in RTL and verified by an automatic UVM-based verification methodology.This RTL implementation describes the logic operations and communication interfaces that are required to perform memory transactions. In this UVM environment, both directed and constrained-random stimuli are used for functionality verification.

Monitors and scoreboards constantly monitor the system behavior and match the output values against the expected results. Coverage metrics are used to measure verification completeness, whereas STA will be done for timing verification purposes.All these activities together form the entire verification methodology flow.

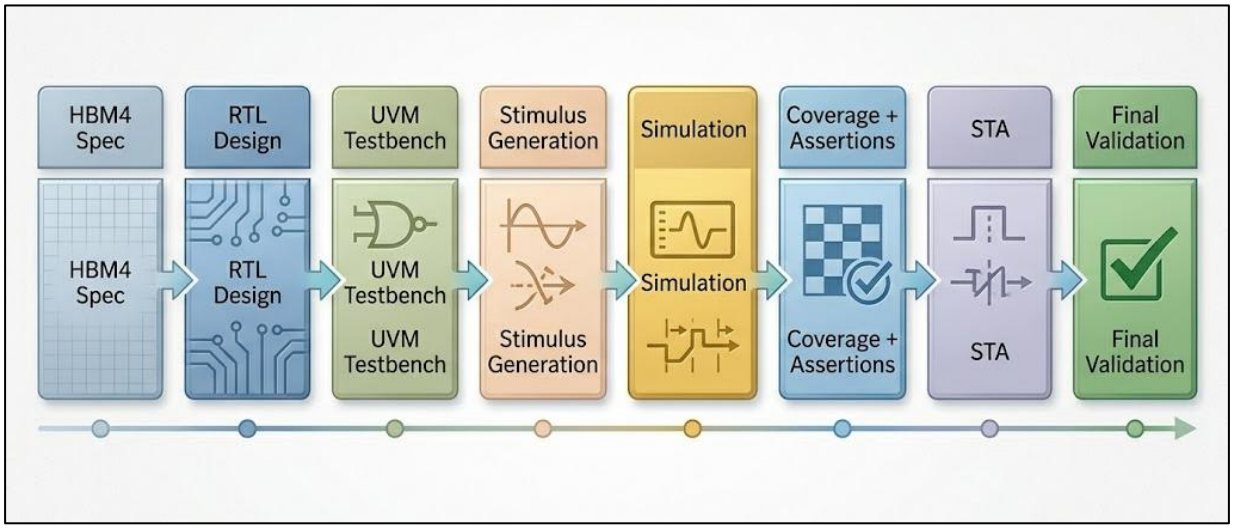


Figure 4 Proposed methodology block diagram

1.6 RESEARCH METHODOLOGY

1. Requirement Analysis and System Specification

Analysed user pain points, manual workflow delays, and security gaps, and defined system components including voice input, supervisor approval handling, Azure Functions, and Cosmos DB containers for user records and reset requests.

2. System Design and Architecture Modelling

Designed the high-level architecture integrating Angular, Azure Functions, and ServiceNow, and modelled Cosmos DB with two collections — *Users* and *PasswordResetRequests* — to support validation and audit logging.

3. System Development

Implemented Azure Function APIs, voice-driven frontend, and ServiceNow callback mechanisms, while configuring Cosmos DB to store employee master data and request-level artifacts such as ticket state and audit logs.

4. Testing and Validation

Verified the full two-step process including user validation, supervisor approval callbacks, password generation, and retrieval, ensuring correct updates to both Cosmos DB collections.

5. Evaluation and Optimization

Analysed execution logs, Cosmos DB audit entries, and service timings to optimize response latency, supervisor callback processing, and error-handling behaviour.

6. Documentation and Future Enhancement Planning

Documented architecture, API flows, ServiceNow configuration, and Cosmos DB data models, and proposed enhancements such as multilingual voice support, behavioural anomaly detection, and end-to-end predictive workflows.

CHAPTER 2

TECHNOLOGY OVERVIEW

2.1 High Bandwidth Memory – Evolution and HBM4

With the increasing demand for more throughput and low latency in computing systems, conventional memory technologies have become highly limited due to the lack of bandwidth and power issues. As a result, an entirely new memory technology called the High Bandwidth Memory (HBM) was developed.

The major advantage of HBM over conventional memory lies in its unique memory packaging technique. Unlike regular memory packaging, which uses wide PCB traces and narrow interfaces, HBM consists of multiple DRAM dies stacked one above another using Through Silicon Vias (TSVs). Communication with such an arrangement of dies occurs via the logic die placed underneath the stack of dies.

Below is how each subsequent generation of HBMs brought improvements:

- HBM (1st Generation): Implementation of stacked memory packaging.
- HBM2: Improvement in bandwidth and storage capacity.
- HBM2E: Improved performance and efficiency in data processing.
- HBM3: Greater bandwidth and scalability of memory.
- HBM4: Expanded interfaces, improved channel organization, and efficient system integration for AI and HPC applications.

HBM4 is expected to operate at extremely fast data rates without increasing power consumption. The use of wider interfaces and more efficient logic organization helps efficiently deal with memory-intensive applications.

In this paper, certain architectural concepts of HBM4 are used at the register-transfer level and verified using verification techniques.ghts how each component contributes to automation, security, and overall operational efficiency.

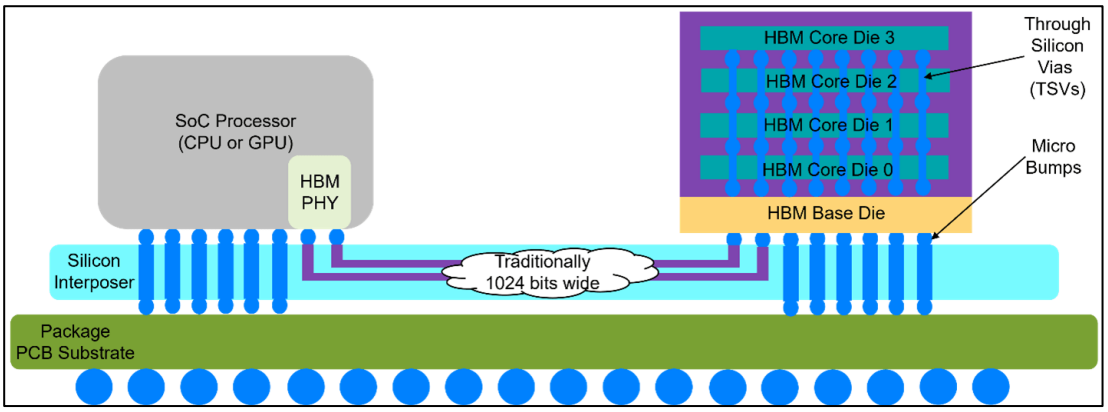


Figure 5 HBM Stack Architecture

2.2 SystemVerilog and RTL Design

SystemVerilog is a Hardware Description Language (HDL) that extends Verilog with verification constructs and other features. It enables modeling of hardware as well as verification, thus becoming the HDL of choice for today's digital ICs design.

The Register Transfer Level (RTL) describes hardware functionality in terms of register actions, combinational logic, sequence, and transfers synchronized by clocking.

Advantages of the RTL approach include:

- Description of digital functionality
- Simulation of circuit functionality
- Verification of operation before synthesis
- Optimization of architecture before implementation

In our current project, the RTL is used to create the architecture of the logic die, control

interface logic, state machine functionality, and communication logic.

Main RTL concepts employed include:

- Clocked sequential logic
- State Machine (FSM) implementation
- Module instantiation with parameters
- Interface communication
- Hierarchical structure

RTL simulation provides functional testing before proceeding to further timing and physical steps of the process.

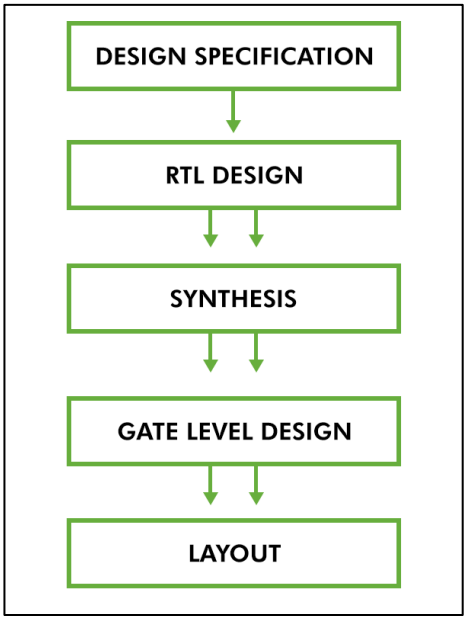


Figure 6 Design Flow

2.3 Universal Verification Methodology (UVM)

With increasing complexity in hardware designs, the conventional method of directed testing fails to ensure comprehensive functionality verification. The Universal Verification Methodology framework offers a standard approach to designing and creating verifiable and scalable testing environments.

UVM consists of a combination of SystemVerilog and features such as automatic test pattern generation, constrained-random verification, transactions-based communication, and modular design testbench.

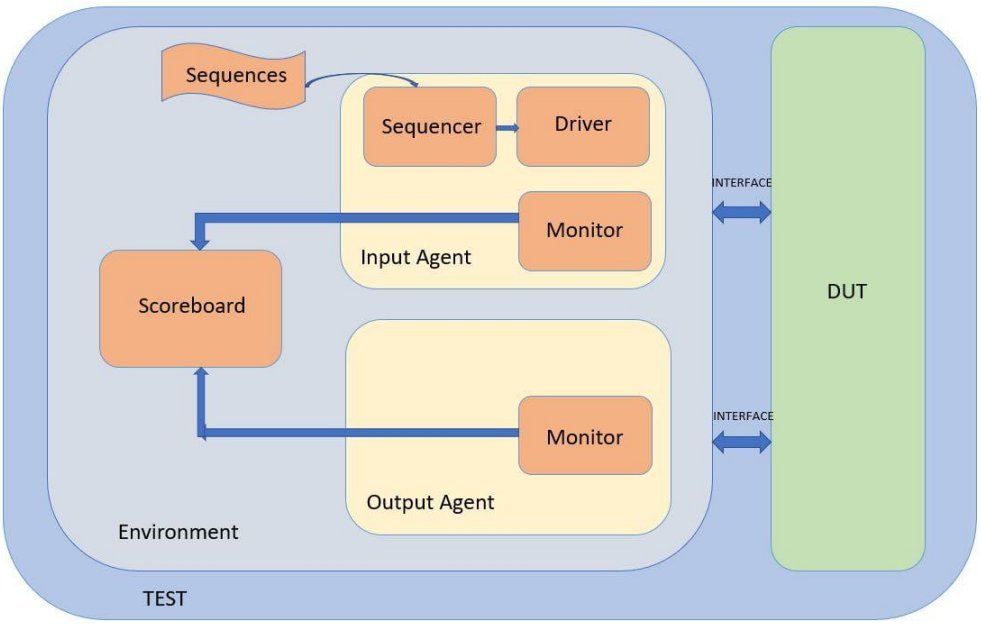


Figure 7 UVM design flow

Some key components in UVM include:

Core Architecture

Test: The controller responsible for configuring the testbench, initializing the environment, and specifying test scenarios.

Environment (Env): The top-level component in the system that includes all sub-components like agents, scoreboards, and coverage collectors.

Interface Agents

Agent: Modular components that group the sequencer, driver, and monitor in order to handle a certain interface/protocol.

Sequence & Sequencer: The sequence is an abstract model of transaction flows, whereas the sequencer controls them.

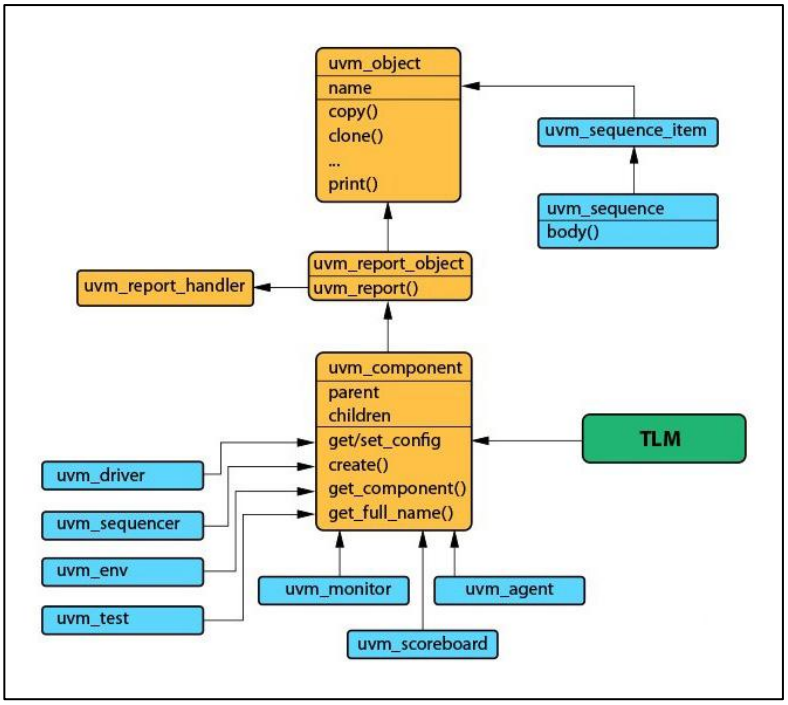
Driver: Extracts transactions from sequences and turns them into physical pin-level waveforms of DUT pins.

Monitor: Captures data generated by the DUT at a physical pin level, samples them, and turns them into packets/sequences.

Analysis & Validation

Scoreboard: Checks captured data, computes its outcome, and checks whether the DUT's actual outputs match the computation.

Coverage Collector: Collects statistics to determine what percent of the verification/design space has been covered.



For this project, a complete UVM environment is developed to validate memory transactions and ensure protocol correctness.

2.4 Simulation and Verification Tools Used

Development and verification of the suggested design based on the HBM4 involved several simulators depending on the particular stage of its implementation. As opposed to usage of a single simulator tool, the different simulators were chosen according to their capabilities in

performing RTL validation, rapid prototyping, and support for a complete set of UVM simulation tasks.

In general, the following steps were taken within the suggested workflow, starting from standalone validation and verification of the RTL, to the final stage of verification simulation.

2.4.1 Quartus Prime for RTL Design and Functional Validation

During the early stage of design development, Quartus Prime was applied for RTL compilation, design and functionality validation, and debugging. The RTL modules designed in SystemVerilog language were compiled and verified regarding their logical operations before integration with the design for verification.

The main operations conducted during using Quartus were the following:

- RTL compilation and syntax validation
- Functional simulation of individual modules
- Verification of interface connection
- Design debugging
- Waveform checking.

2.4.2 EDA Playground for Initial UVM Development

EDA Playground acted as a cloud-based development environment that was used during the process of developing the initial UVM framework. The use of the tool helped in the quick development and debugging of UVM-related items without having to set up the entire local simulation environment.

EDA Playground was highly valuable during the early stages of developing the verification environment and the debugging of UVM component interactions.

Some of the tasks carried out through EDA Playground include:

- Development of the basic testbench framework in UVM
- Testing of transactions flow

- Driver and Monitor debugging
- Sequence execution
- Testing of UVM connectivity problems

2.4.3 QuestaSim for Comprehensive UVM Verification

After basic verification, QuestaSim was employed as the main simulation tool for running comprehensive UVM tests and assessing the correctness of the entire design.QuestaSim offered advanced debugging functionality, as well as comprehensive support for SystemVerilog and UVM verification methodologies. The tool allowed performing constrained-random testing and analyzing the behavior of the simulation.

Verification was conducted using the following tasks within QuestaSim:

- Comprehensive UVM environment test
- Constrained-random verification
- Coverage generation and analysis
- Assertion checking
- Debugging using waveform
- Scoreboard verification
- Transaction-level debugging

QuestaSim was utilized as the final simulator for protocol evaluation and verification.

CHAPTER 3

SYSTEM DESIGN & ARCHITECTURE

3.1 INTRODUCTION TO SYSTEM DESIGN

The High Bandwidth Memory (HBM) uses vertically stacked DRAM dies which are interconnected using TSVs and controlled using a special logic die. While traditional memories operate using narrow high frequency buses, HBM employs extremely wide but low-frequency interfaces along with short connection distances. The presented project implements a simple bank model following the principles of HBM4 with an emphasis on transaction completion, command processing, timing check, and DDR-like data transfer operations.

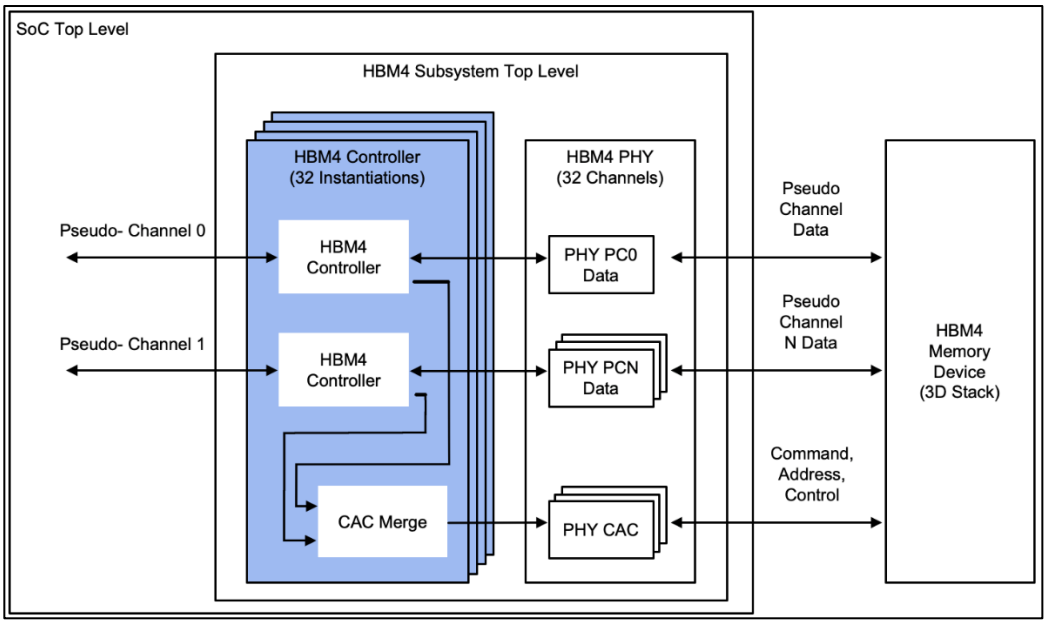


Figure 8 Proposed HBM4 Functional Architecture

There are two types of main memory transactions implemented by the architecture Write transaction & Read transaction. Both transactions go through a predetermined command sequence, providing proper timing relations between commands and data exchange operations. The architecture, developed for this project, includes:

- Command processing unit
- Memory array

- Bank control finite state machine
- Timing control
- PHY wrapper
- Data transfer unit
- DQS generator
- Readback interface

3.2 LOGIC DIE

The Role of logic die as the controlling layer to coordinate the communication from external command to internal memory operations. Function of the logic die

Command Reception :

- start
- op
- row_addr
- col_addr
- data_to_write

Where start triggers the transaction and op specifies either the read or write operation.
The architecture is designed to ensure efficient routing of operations, minimal latency, high throughput, and full compliance with security standards.

Timing parameter

Write Latency (WL): The specific time delay in terms of clock cycles between when a write command is issued and when the data arrives at the HBM4 2048-bit wide data bus.
Read Latency (RL): Time delay between a read command being issued and the initial data being sent out via the data bus from the HBM4 stack.

Row-to-Column Delay (t{RCD}): Time delay between when a particular memory row is opened (activated) using an ACTIVATE command and when a read/write column command may be issued.

Write Recovery (t{WR}): Important time delay between when the data burst of a write operation ends and the PRECHARGE command for closing the row can be issued.

Read-to-Precharge (t{RTP}): Time delay between a read command being issued and the PRECHARGE command to release the row from the read state.

Precharge Delay (t{RP}): Minimum time needed for a row to be deactivated and closed before another row within the same memory bank can be accessed.

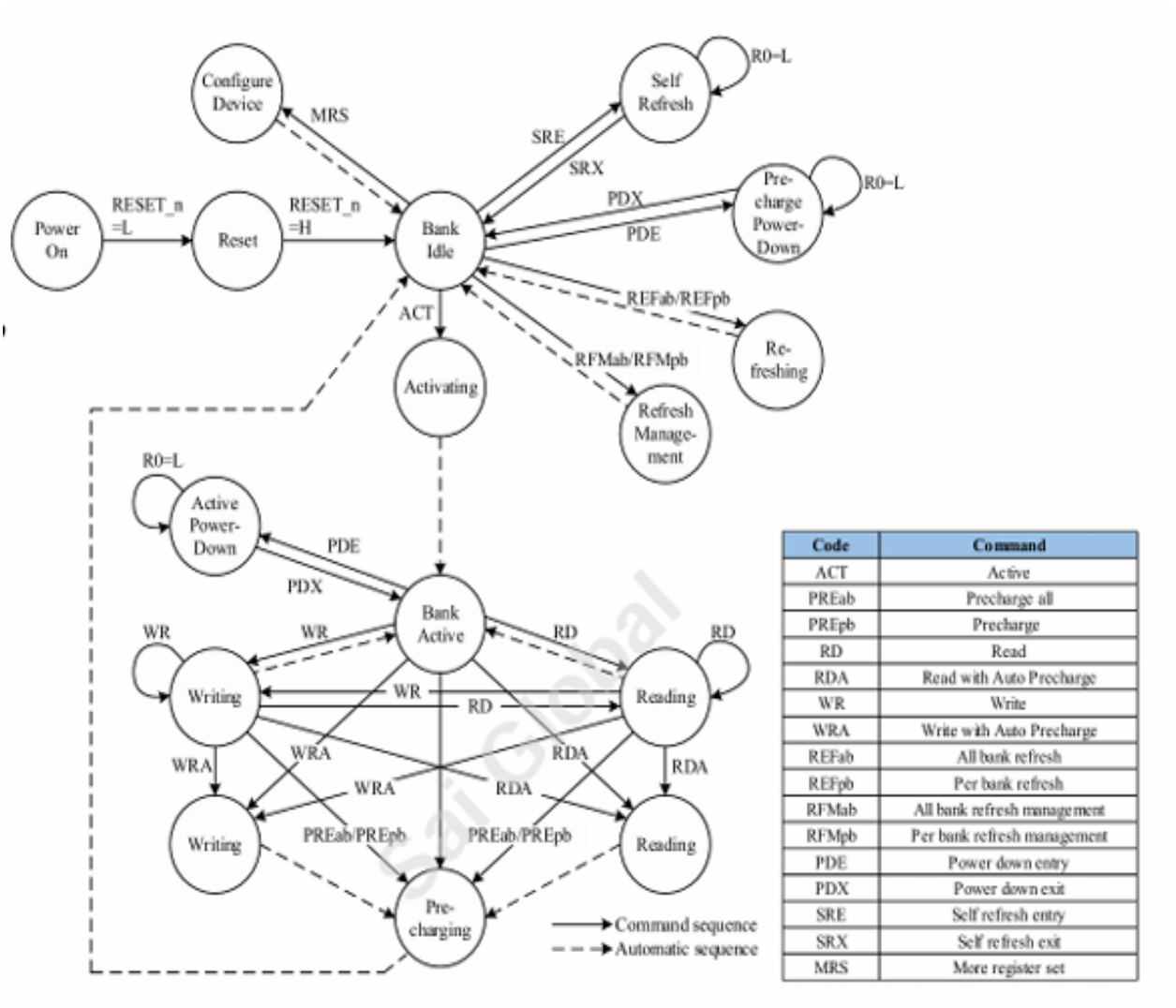
Data Transfer Management

During write:

- Incoming burst data stored internally
- Memory updated at valid timing point

During read:

- Memory contents loaded
- Burst output generated through DQ bus



Code	Command
ACT	Active
PREab	Precharge all
PREpb	Precharge
RD	Read
RDA	Read with Auto Precharge
WR	Write
WRA	Write with Auto Precharge
REFab	All bank refresh
REFpb	Per bank refresh
RFMab	All bank refresh management
RFMpb	Per bank refresh management
PDE	Power down entry
PDX	Power down exit
SRE	Self refresh entry
SRX	Self refresh exit
MRS	More register set

HBM4 Bank State Machine Simplified

Figure 9 HBM 4 BANK STATE MACHINE (reference - jedec)

3.3 INTERFACE SIGNAL SPECIFICATION

The interface provides command, timing, strobe, and data communication

CLOCK SIGNALS

Signal	Description
CK_t	Primary clock
CK_c	Complementary clock
clk_2x	Double-frequency data clock

COMMAND SIGNALS

Signal	Function
start	Begins transaction
op	Read/Write selection
row_addr	Row selection
col_addr	Column selection

DATA SIGNALS

Signal	Description
DQ	Data bus
WDQS	Write data strobe
RDQS	Read data strobe

OUPUT SIGNALS

Signal	Description
done	Transaction complete
rd_valid	Read output valid

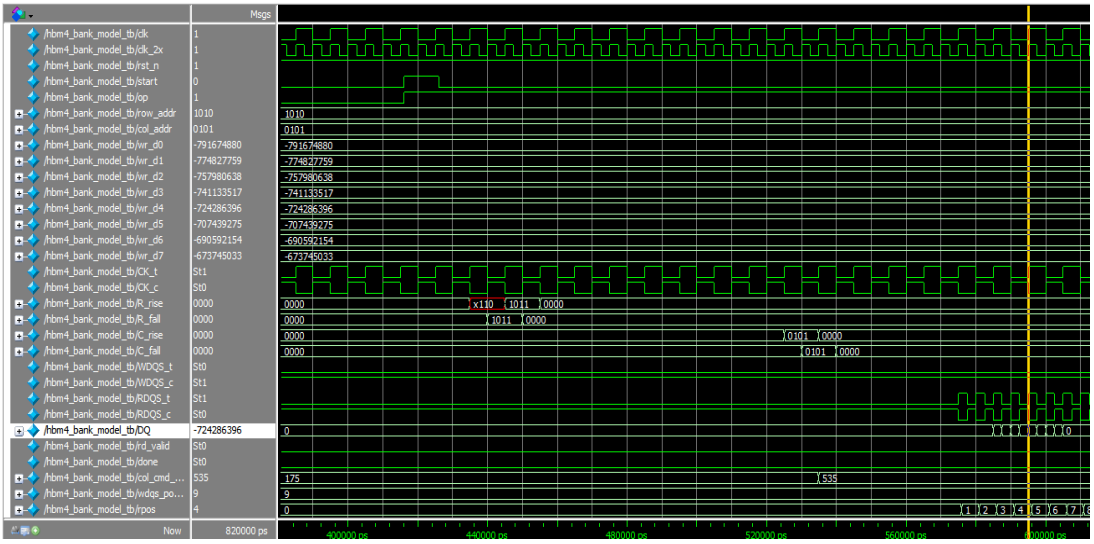


Figure 10 Waveform

3.4 DESIGN PARAMETER

The proposed architecture uses configurable parameters to enable flexibility.

Parameter	Description	Default
WL	Write latency	4
RL	Read latency	6
BL	Burst length	8
tRCD	Activate delay	7
tWTR	Write to read delay	4
tWR	Write recovery	6
tRTP	Read precharge	4
tRP	Precharge delay	7
MEM_DEPTH	Memory depth	16
DQ_W	Data width	32

3.5 BANK STATE MACHINE DESIGN

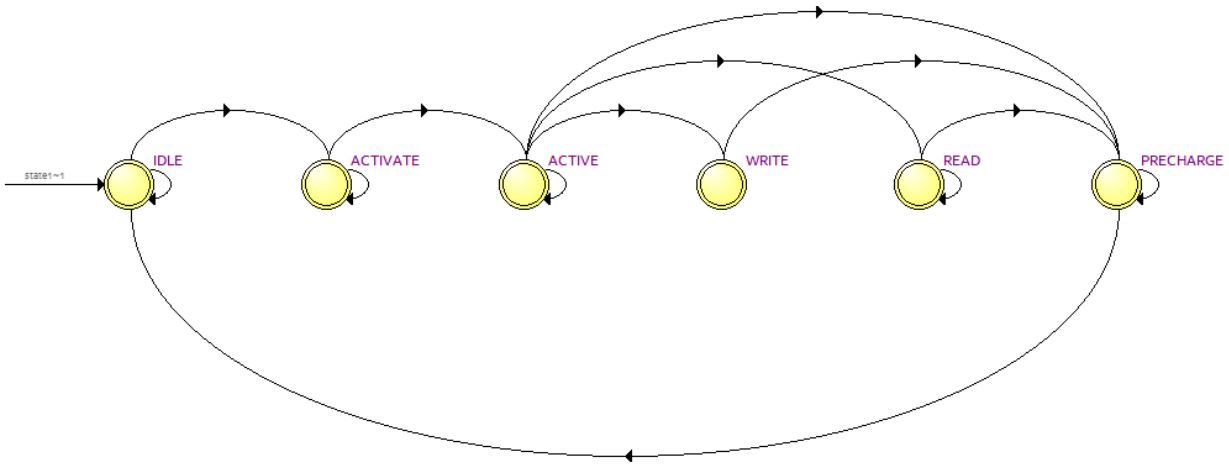


Figure 11 bank state machine

IDLE: The default standby state where the entire bank memory is closed and precharged. The controller stays in this state while watching the command queue until a request for accessing a new row is received before moving on to perform any state transitions.

ACTIVATE: The state that occurs after issuing the activate command along with a bank and row address by the controller. This step is used to physically transition the data from the target row to the sense amplifiers inside the bank memory.

ACTIVE: The active state which happens after the row-to-column delay (t_{RCD}) completes. This is because the memory row is now fully opened and stabilized acting as the hub which can receive further column commands to be read or written.

WRITE: The data-input state is reached after executing a write command to input data to the HBM4 data bus having a 2048-bit width.

READ: This is the state that involves fetching the information stored in the active row that is open at the moment. Data from the memory row requested is sampled by the HBM4 stack and placed on the data lines, making the data ready for use by the controller right after the specified Read Latency (RL).

PRECHARGE: The final stage, in which the controller makes the active row inactive again while resetting the voltage of the bitline back to its normal state.

3.6 PHY WRAPPER

The PHY wrapper converts internal memory operations into bus-level signaling.

Its responsibilities include:

- Clock forwarding
- DQS generation
- DQ control
- Timing synchronization

The design generates:

- WDQS during write
- RDQS during read

The DQ output is enabled only during active transfer windows.

The implementation supports DDR behavior by using the doubled-frequency clock and controlled pair indexing for burst generation.

3.7 RTL SCHEMATIC

The Architectural Design & Architecture The architecture design has adopted the concept of hierarchical, modular architecture that emphasizes the rigid separation between the data and control paths to enhance efficiency and throughput while minimizing latency.

Control Path Components

FSM Control: The brain of the design and is in charge of executing state transition activities and ensuring the adherence of protocol requirements, among others.

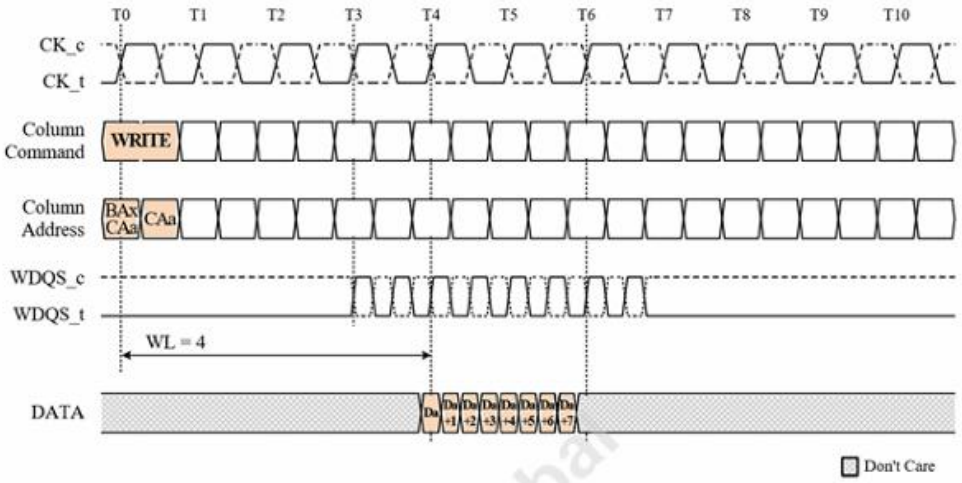
Address Latch: Used to stabilize and maintain the validity of addresses of row, column, and bank throughout the entire memory operation phase.

Timing Control: Strict hardware timing conditions like t_{RCD} and t_{RP} latencies are enforced to avoid bus conflict and design instability.

Command Generator: Translates internal control states to hardware commands (ACTIVATE, READ, WRITE, PRECHARGE, etc.) as per the memory specifications.

6.3.3.3.3 Write Operation

Single write bursts are shown in Figure 44.

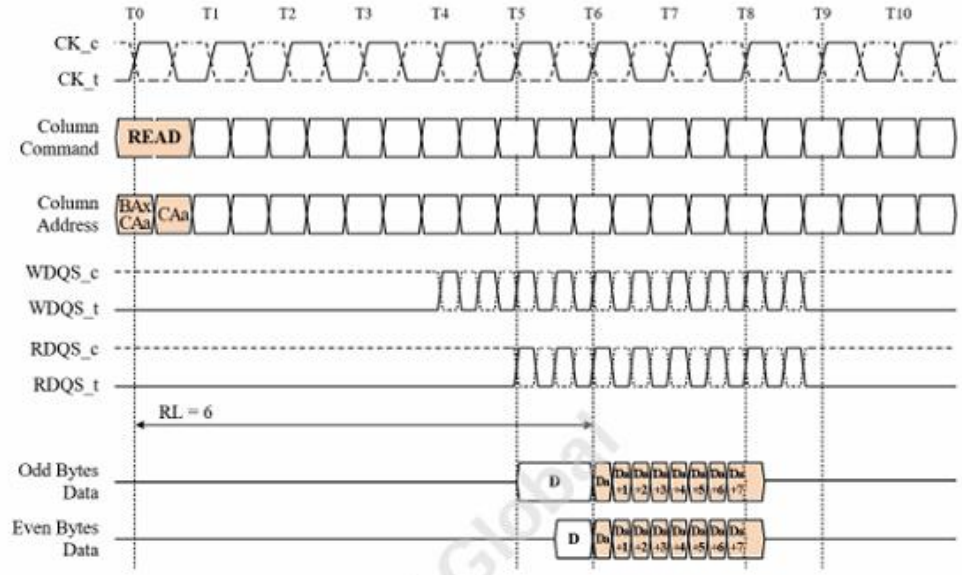


NOTE 1 BAx = bank address x; CAa = column address a.
NOTE 2 WL = 4 is shown as an example.
NOTE 3 DATA = DQ[31:0]. DBI[3:0], ECC[1:0] for PC0, and DQ[63:32], DBI[7:4],ECC[3:2] for PC1.
DPAR = DPAR 0 for PC0 and DPAR1 for PC1 (if applicable).
WDQS_t/_c is WDQS0_t/_c for PC0, and WDQS1_t/_c for PC1.
NOTE 4 Da...Da+7 = data-in for WRITE command a.
NOTE 5 t_{DQSS} = 0 is shown for illustration purposes.
NOTE 6 WDBI could be on or off and is controlled with MR0 OP1.

Figure 12 single write operation (reference - jedec)

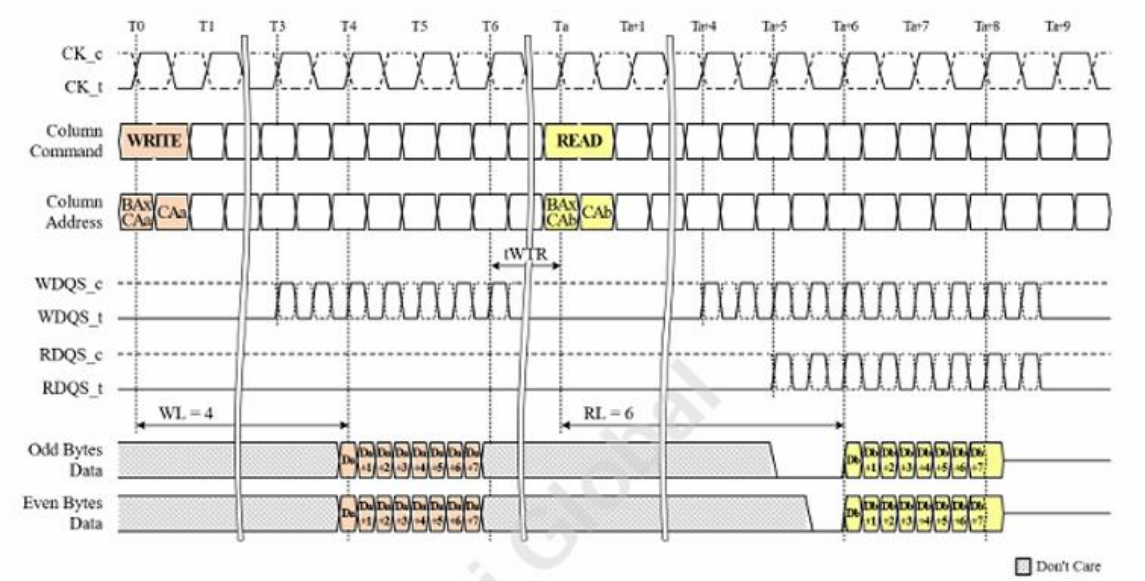
6.3.3.2.4 Read Operation

Single read bursts are shown in Figure 36 for BL=8.



NOTE 1 BAx = bank address x; CAa = column address a.
NOTE 2 RL = 6 is shown as an example.
NOTE 3 DATA = DQ[31:0]. DBI[3:0], ECC[1:0]. SEV[1:0] for P C0, and DQ[63:32], DBI[7:4],ECC[3:2]. SEV[3:2] for PC1.
WDQS_t/_c is WDQS0_t/_c for PC0, and WDQS1_t/_c for PC1.
RDQS_t/_c is RDQS0_t/_c for PC0, and RDQS1_t/_c for PC1.
NOTE 4 Da...a+7 = data-out for READ command a.
D = last data-out from previous READ command (not if first REA D after reset, MRS, self refresh or write-to-read).
NOTE 5 $t_{WDQS2DQ\_O}$ = 0 and nominal t_{QW} is shown for illustration purposes.
NOTE 6 RDBI could be on or off and is controlled with MR0 OP0.

Figure 13 single read operation (reference - jedec)



NOTE 1 BAx = bank address x; CAa,b = column address a,b.
NOTE 2 WL = 4 and RL = 6 are shown as examples.
NOTE 3 DATA = DQ[31:0], DBI[3:0], ECC[1:0], SEV[1:0] for PC0, and DQ[63:32], DBI[7:4], ECC[3:2], SEV[3:2] for PC1.
DPAR = DPAR0 for PC0 and DPAR1 for PC1 (if applicable).
WDQS_t/_c is WDQS0_t/_c for PC0 and WDQS1_t/_c for PC1.
RDQS_t/_c is RDQS0_t/_c for PC0 and RDQS1_t/_c for PC1.
NOTE 4 Da...Da+7 = data-in for WRITE command b. Db...Db+7 = data-out for READ command a.
NOTE 5 $t_{WDQS2DQ\_O}$, t_{DQSS} = 0 and nominal t_{QW} is shown for illustration purposes.
NOTE 6 $t_{WTR} = t_{WTRL}$ when both WRITE and READ access banks in the same bank group, otherwise $t_{WTR} = t_{WTRS}$.
NOTE 7 WDBI could be on or off and is controlled with MR0 OP1.
NOTE 8 READ operation shown with RDBI enabled. RDBI is enabled/disabled with MR0 OP0.

Figure 14 back to back write to read operation

Correct command sequence

- Timing-based data transfer
- Accurate bursts
- Transaction completion

Path and Interface Elements

Memory Core: Serves as the target memory array type wherein the physical realization of the data transactions is performed.

Burst Engine: Handles burst data transfer rates through auto-increment of column addresses and proper data framing.

DQS Generator: Produces essential data strobe (DS) signal pairs (called DQS) necessary for correct timing in edge or center aligned data capture.

Output Interface: Conditions and drives the final data (DQ) signals and control bus signals

to and from the pins.

Design Verification Process

A systematic verification approach starts from synthesis validation and ends in full dynamic simulation.

Burst beats number eight and are observed for pass or fail

1. Functional Compilation & Synthesis

Quartus Validation: The Intel Quartus Prime tool is employed in the first step to carry out syntax validation, static functional compilation, and structural resource computation to ensure synthesizability prior to proceeding with advanced simulation environments.

2. Simulation Key Areas

The dynamic behavioral simulation is designed to confirm the following vital function vectors:

Protocol Compliance: To prove that the command generator outputs the exact sequence as per the protocol specification.

Timing Determinism: Ensure data transactions occur only in accordance with the specified latency for Read and Write and inter-command delay period.

Burst Length Functionality: Proves that subsequent burst cycles operate uninterrupted without affecting neighboring memory bank or data frame transactions.

Transaction Execution: End-to-end transaction operation validation from idle to activation, data and eventually precharge stages.

3. Testbench Functions & Verification Approach

A verification testbench has been developed to effectively validate system performance/reliability in terms of the following procedures:

Functional Read/Write Operation: Carry out successive read/write transactions to confirm

data update and access operations.

Cycle Accurate Timing: Check physical lines' signals for setup/hold violations and any timing error in the range of frequencies.

Data Integrity & Scoreboarding : Perform automated end-to-end integrity tests of data stimulus against received output packets.

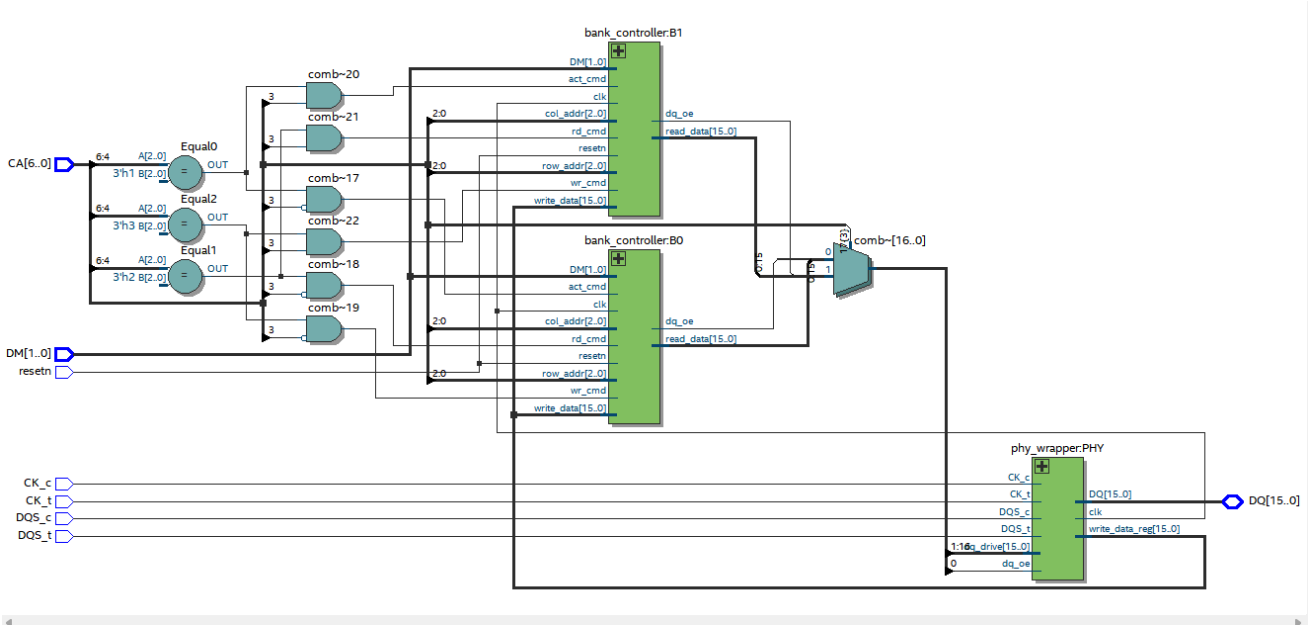


Figure 15 RTL netlist view

CHAPTER 4

UVM TESTBENCH ARCHITECTURE

4.1 OVERVIEW OF THE UV ARCHITECTURE

The In order to prove the working ability of the proposed HBM4 bank module, a testbench was built using the Universal Verification Methodology. The testbench has a modular structure where stimulus generation, stimulus application, monitoring, checking, and coverage collection are kept separately. The verification is done at transaction level with the help of an interface designed using SystemVerilog. The process starts with test generation, then followed by sequence generation, signal application, transaction monitoring, result checking, and coverage collection.

The environment was designed to support:

- Directed verification
- Constrained random testing
- Functional checking
- Coverage-driven validation
- Assertion-based protocol verification
- Reusable test execution

The top-level environment contains:

- Sequence library
- Driver
- Monitor
- Agent
- Scoreboard
- Coverage collector
- Environment controller
- Test library

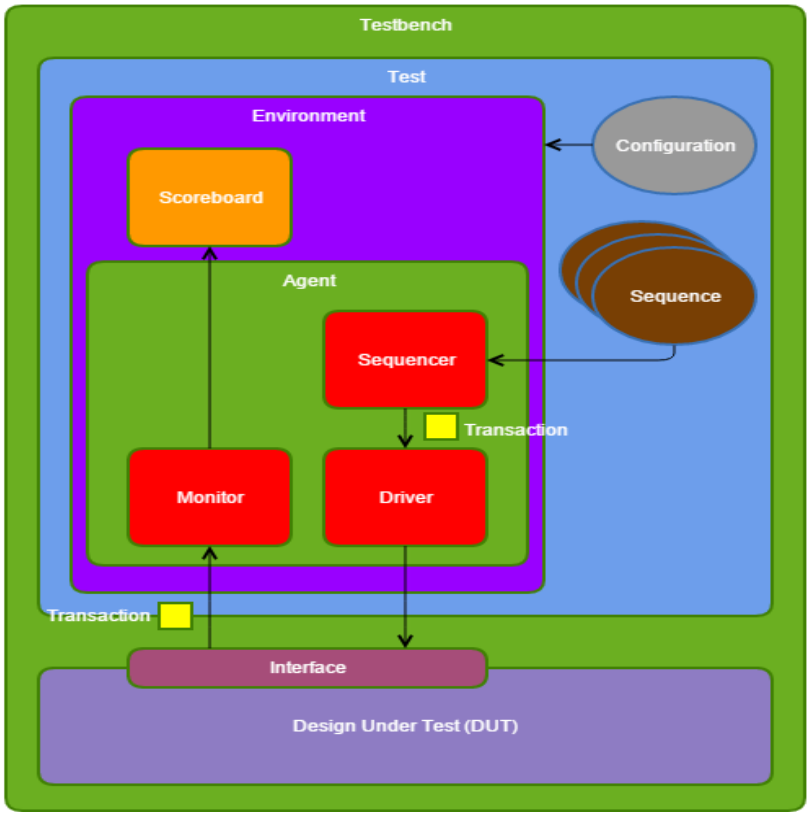


Figure 16 Complete uvm architecture

4.2 TRANSACTION (hbm4_item)

The frontend is Transaction object is the basic communication primitive within the UVM testbench environment. hbm4_item class represents memory transactions that will be passed across the entire verification environment.

Transaction consists of the following fields:

- Type of operation (read/write)
- Row address
- Column address
- Write burst data
- Read burst data
- Data mask info

Transaction abstraction facilitates generation of stimuli independent of signal implementation. Transaction modeling reduces complexity of verification tasks and provides scalability for further protocol expansion.

4.3 SEQUENCE LIBRARY (hbm4_seq_lib)

Sequence Library creates verification tests and manages transactions flow. Different sequences have been implemented in order to verify various functionalities of the memory.

Some of the implemented sequences are:

- **Write-Read Directed Sequence:** Verifies correct write/readback operation.
- **Address Sweep Sequence:** Conducts transactions using several rows and columns.
- **Pattern Sequence**: Uses several data patterns to find potential logic problems.
- **Randomized Sequence:** Generates random transactions
- **Assertion-Oriented Sequences**: Tests certain conditions for particular protocols.

The sequence level offers high test reuse without modifications of low-level verification modules.

4.4 DRIVER (hbm4_driver)

The driver is responsible for converting object-level transactions into operations on the DUT interface. Transactions are taken from the sequencer by the driver and driven via the virtual interface using clocked blocks.

Responsibilities of the driver:

- Generating start pulses
- Driving operation selection
- Sending addresses
- Driving write burst data
- Waiting for transaction completion
- Synchronization

Particular care was paid to synchronizing time to avoid protocol violations.This is achieved with the help of the clocking block of the interface.These ensure high-speed data lookups, essential during live approval and retrieval operations.

4.5 MONITOR (hbm4_monitor)

Monitor passively monitors the behavior of the DUT and constructs transactions.Unlike driver, the monitor does not produce any signal. It records the interface activities and translates the low-level outputs into transaction objects.

The observed signals are:

- start
- op
- row_addr
- col_addr
- rd_valid
- rd_data
- done

Captured data is then sent to:

- Scoreboard
- Coverage collector

Monitor provides an independent monitoring of DUT's behavior.

4.6 AGENT (hbm4_agent)

Agent is used to hold all of the components for verification.

The implemented agent includes:

- Sequencer
- Driver
- Monitor

Such approach makes the setup of environment simpler and improves its reuse.For communication between the sequencer and driver, transaction ports are utilized, whereas transactions are published by the monitor.

4.7 ENVIRONMENT (hbm4_env)

Environment comprises the entire verification infrastructure.

Environment includes:

- Agent
- Scoreboard
- Coverage collection

The transactions made during execution are redirected to check and coverage.

Responsibilities of environment include:

- Creation of component
- Connection handling
- Distributing configuration
- Test orchestration

This design provides better maintenance.

4.8 SCOREBOARD (hbm4_scoreboard)

The Scoreboard for Functional Correctness Verification Your solution constructs an internal memory to hold references and compare read responses with the desired values.

Verification steps:

- Write:Store references
- Read:DUT response comparison & pass/fail determination
- Gathered metrics: Writes, Reads, Passes , Fails

The scoreboard automatically produces summary reports post-simulation.This approach allows you to perform automatic verification without any wave viewing.

4.9 COVERAGE COLLECTOR (hbm4_coverage)

The coverage collector measures verification completeness.Your implementation contains extensive functional coverage including:

Operation Coverage

- Read
- Write

Address Coverage

- Row coverage
- Column coverage

Data Mask Coverage

- All mask combinations

Data Pattern Coverage

- All zeros
- All ones
- Alternating patterns
- Checkerboard patterns
- Walking ones

Signal Toggle Coverage

- start
- done
- reset
- WDQS
- RDQS

Assertion Observation Coverage

Protocol assertions are tracked to ensure scenarios are exercised.

4.10 TEST LIBRARY

The test library organizes reusable verification scenarios. Implemented tests include:

Test	**Purpose**
Base Test	Environment creation
Write–Read Test	Functional validation
Address Sweep Test	Address coverage
Data Pattern Test	Pattern verification

The base test manages:

- Reset handling
- Environment startup
- Report generation

The derived tests extend functionality through sequence execution.

CHAPTER 5
IMPLEMENTATION

5.1 RTL IMPLEMENTATION DETAILS

The automated An HBM4-based memory controller was created using SystemVerilog RTL coding. Design considerations included modeling of memory transactions, timing control, bursts, and protocol signals.The RTL architecture was created as parameterized blocks allowing changes in memory timing, burst configuration, and data width without a need for reimplementation of an entire architecture.

There are two main functions supported by the memory controller:

- Write transaction
- Read transaction

Both transactions have a defined timing-aware execution process managed by a finite-state machine. The blocks used in the RTL implementation were:

Transaction Controller: Manages memory transactions and initiates them once the start signal is received.

Functions:

- Transaction initiation
- Command management
- Address management

Memory array: Provides internal memory using parameterized depth.

Functions:

- Data storage
- Burst extraction

- Memory readback

Timing Controller: Controls the memory according to the protocol timing requirements.

Implemented timing parameters include:

- Write Latency (WL)
- Read Latency (RL)
- Row to Column Delay (tRCD)
- Write Recovery (tWR)
- Read to Precharge (tRTP)
- Precharge Delay (tRP)

Data Engine: Performs data bursts and operates according to DDR standards.

Functions:

- Burst management
- DQ generation
- Read/Write operation management

The RTL was initially tested using Quartus simulator and then moved to UVM verification..

5.2 UVM COMPONENT IMPLEMENTATION

The verification framework was implemented using UVM to provide reusable and scalable verification. The implementation followed layered abstraction where transactions are generated independently from DUT signaling.

Implemented components include:

Component	**Implementation Purpose**
hbm4_item	Transaction abstraction
hbm4_seq_lib	Stimulus generation
hbm4_driver	Signal application
hbm4_monitor	Observation
hbm4_agent	Component grouping
hbm4_env	Environment integration
hbm4_scoreboard	Result checking
hbm4_coverage	Coverage collection
hbm4_test_lib	Test execution

Component implementation followed UVM phases:

- build_phase()
- connect_phase()
- run_phase()
- report_phase()

The modular architecture improved debugging efficiency and verification reuse.

5.3 DIRECTED TESTBENCH FLOW

Directed testing was implemented to validate deterministic scenarios before executing randomized verification.The objective of directed testing was to confirm:

- Functional correctness

- Interface connectivity
- Timing compliance
- Memory operation sequence

The directed verification flow included:

Write Test

Sequence:

Reset
↓
Start
↓
Activate
↓
Column Command
↓
Data Transfer
↓
Precharge
↓
Done

Expected write data was stored for later comparison.

Read Test

Sequence:

Read Command
↓
Memory Access
↓
Burst Output
↓
Comparison
↓
PASS / FAIL

The scoreboard automatically compared returned data against stored expected values. Directed testing allowed rapid debugging before enabling random verification.

5.4 CONSTRAINED RANDOM STIMULUS GENERATION

Once we validated our deterministic behavior, constrained random verification was utilized for improving the scenario coverage.

Randomization was performed on:

- Operation selection
- Address generation
- Data patterns
- Transaction order

Constraints were enforced in order to avoid any invalid scenario execution and protocol function validity.

What was achieved:

- More functional coverage
- Finding out corner cases
- Decreasing manual test creation effort

Random stimulus generation was done via sequence library under control of UVM randomization.

Coverage criteria were used for deciding when to finish. The implementation approach set up a comprehensive design and verification flow starting with RTL design up to automation of UVM verification, constrained-random testing, assertions, and coverage. Simulation and verification results achieved through this implementation approach are elaborated in the next chapter.

CHAPTER 6
RESULTS & DISCUSSION

6.1 INTRODUCTION

this chapter discusses the outcomes of verification conducted upon implementation of the RTL and UVM verification environment designed for the HBM4-inspired memory architecture. The functional correctness was verified via directed and constrained-random verification, and completeness of verification process was ensured via coverage and scoreboards verification.As can be observed from the results presented in this chapter, proper execution of read/write operations is achieved.

6.2 DIRECTED TB WAVEFORM ANALYSIS

Initial directed tests were done to verify the correctness of deterministic memory behavior in terms of proper command ordering, windowing, and burst transfers.

- The tests verified the following:
- Proper command generation
- Correct execution of write and read transactions
- Proper DQS timing
- Correct DQ bursts
- Completion of transactions

Based from the waveforms, it can be seen that the memory controller performs back-to-back write and read transactions with no protocol violations.

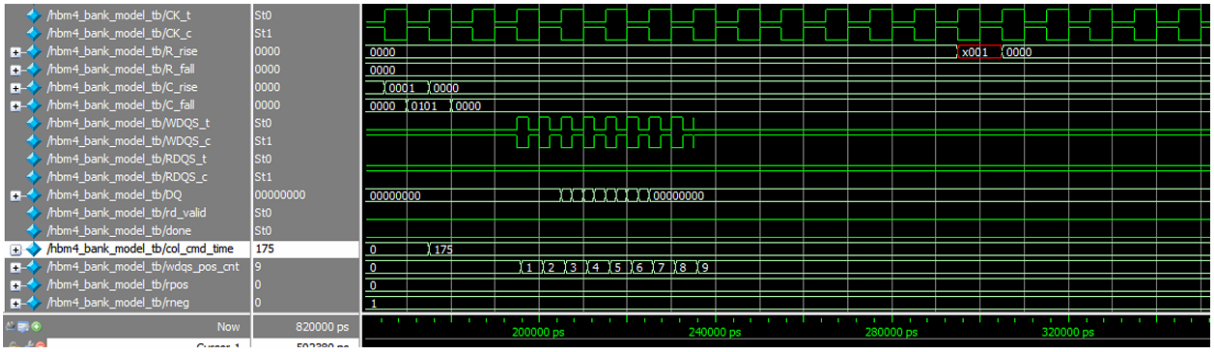


Figure 17 Waveform executes write and read operation

In write transaction,

- Activation command sent

- Column command sent

- WDQS goes high and DQ bursts starts

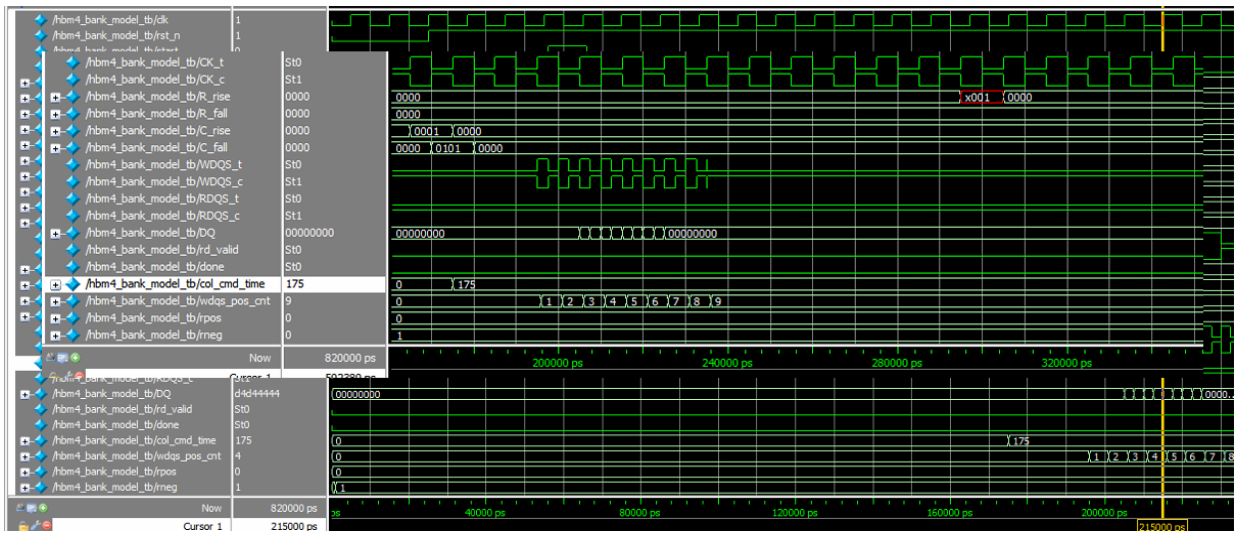


Figure 18 Complete waveform of back-to-back write and read operation (write operation)

In read transaction,

- Read command sent

- RDQS goes high

- DQ burst containing stored

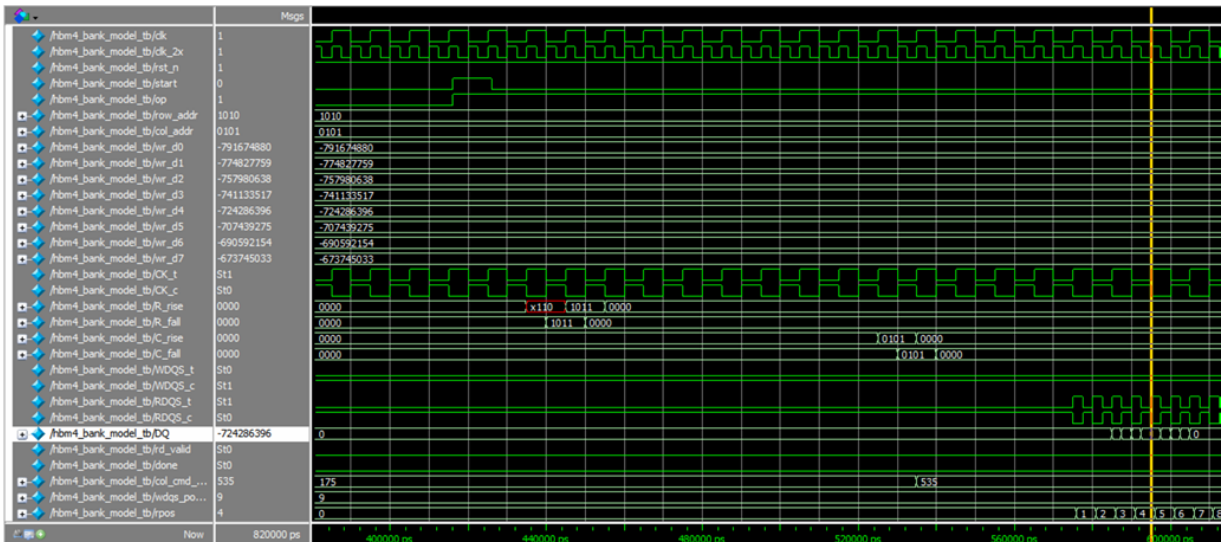


Figure 19 Complete waveform of back-to-back write and read operation (read operation)

6.3 FUNCTIONAL VERIFICATION RESULTS

Functional verification was executed using the UVM environment implemented in QuestaSim. Simulation logs indicate successful execution of verification scenarios.

Summary obtained:

Metric	**Result**
UVM INFO	1842
UVM WARNING	1
UVM ERROR	0
UVM FATAL	0

The absence of errors and fatal messages indicates that the DUT operated correctly under executed test conditions.

Component execution summary indicates active participation of:

- Driver
- Monitor
- Sequence library
- Scoreboard
- Coverage collector

The generated reports confirm stable transaction execution and successful integration of verification components.

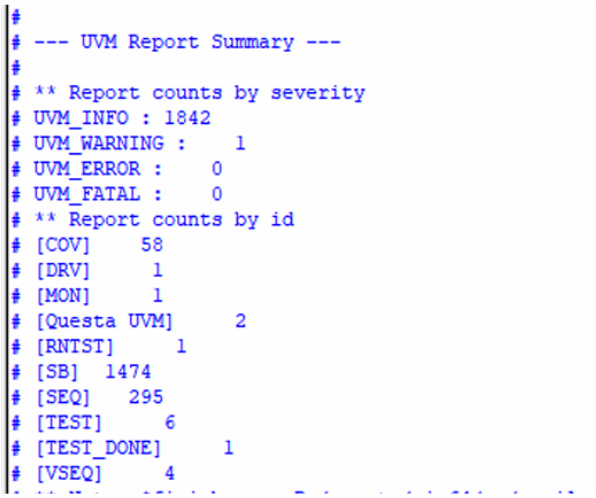


Figure 20 UVM SUMMARY

6.4 COVERAGE REPORT

Coverage analysis was performed to evaluate verification completeness.

Overall verification results achieved:

Metric	**Coverage**
Overall Coverage	**95.85%**
Statement Coverage	**100.00%**
Branch Coverage	**98.46%**
Toggle Coverage	**98.50%**
FSM State Coverage	**100.00%**
FSM Transition Coverage	**100.00%**
Assertion Coverage	**71.42%**

The achieved coverage demonstrates that nearly all functional scenarios and execution paths were exercised.

Coverage Summary by Type:

Total Coverage:					95.58%	93.55%
Coverage Type	**Bins**	**Hits**	**Misses**	**Weight**	**% Hit**	**Coverage**
Covergroups	376	332	44	1	88.29%	**95.85%**
Directives	5	5	0	1	100.00%	**100.00%**
Statements	119	119	0	1	100.00%	**100.00%**
Branches	65	64	1	1	98.46%	**98.46%**
FEC Conditions	19	16	3	1	84.21%	**84.21%**
Toggles	800	788	12	1	98.50%	**98.50%**
FSMs	14	14	0	1	100.00%	**100.00%**
States	7	7	0	1	100.00%	**100.00%**
Transitions	7	7	0	1	100.00%	**100.00%**
Assertions	7	5	2	1	71.42%	**71.42%**

Figure 21coverage report

6.5 COVERGROUP ANALYSIS

Functional coverage was further analyzed through covergroups implemented inside the UVM environment.

Covergroup results:

Parameter	**Result**
Total Bins	376
Covered	332
Missed	44
Coverage	95.85%

Coverage included:

- Operation types
- Address combinations
- Data patterns
- Transaction ordering
- Protocol conditions

The missed bins mainly correspond to rarely occurring combinations generated under constrained conditions.

Coverage Summary By Instance:

Scope	TOTAL	Cvg
TOTAL	95.85	95.85
hbm4_pkg	95.85	95.85
hbm4_coverage	95.85	95.85

Local Instance Coverage Details:

Total Coverage:					88.29%	95.85%
Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
Covergroups	376	332	44	1	88.29%	95.85%

Recursive Hierarchical Coverage Details:

Total Coverage:					88.29%	95.85%
Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
Covergroups	376	332	44	1	88.29%	95.85%

Figure 22 Functional covergroup statistics

6.6 SCOREBOARD PASS/FAIL SUMMARY

The scoreboard was responsible for validating DUT outputs against expected memory responses.

Verification flow:

Write: Expected values stored

Read: Returned values captured

Comparison: PASS/FAIL generated

Observed results:

Parameter	**Result**
Data Comparison	PASS
Burst Integrity	PASS
Readback Accuracy	PASS
Protocol Validation	PASS

No mismatches were observed during executed transactions.This confirms:

- Correct internal memory updates
- Accurate burst reconstruction
- Reliable readback operation

Figure 23 Scoreboard comparison showing successful burst validation.

6.7 OBSERVATION AND INFERENCES

The From waveform analysis, coverage evaluation, and scoreboard check results, the following can be concluded:

- RTL properly performed both read and write transactions.
- FSM transition was fully carried out.
- Burst transfer timing corresponded to the latency provided.

- UVM modules behaved properly.
- Coverage ratio was above 95%.
- No protocol violation occurred.

However, assertion coverage was less than that of functional coverage, and further improvement is possible. It should be noted that the proposed HBM4 verification methodology was successful in verifying the functional behavior of the memory controller and proving the efficiency of UVM-based verification for transaction-oriented memory systems.

CHAPTER 7

CHALLENGES & ENHANCEMENTS

7.1 TIMING CONSTRAINTS CHALLENGES

One of the problems faced during the design process was related to the proper alignment of the timing of memory transactions without compromising the protocol behavior. Given the fact that the design was required to mimic the HBM4-based operation, many timing aspects needed to be considered. For example, the timing values such as write latency (WL), read latency (RL), row to column delay (tRCD), write recovery (tWR), read to precharge delay (tRTP), and precharge delay (tRP) created dependence between state transitions and data transfers.

Initially, there seemed to be a problem with the timing of data events and command events, as command generation occurred separately from burst transfer timing during the process.This caused the situation when DQS activation and DQ transfer windows were not aligned properly. To resolve this problem, the values of timing were parametrized and included in the transition logic between states. The verification process involved waveform checking and numerous simulations.

7.2 COVERAGE CLOSURE ISSUES

Having adequate coverage was one more important difficulty to be overcome during the process of developing the solution.

Even though the first created environment passed all test sessions, several coverages were still left without visits because of not enough diverse scenarios. The early tests were performed using deterministic sequences, which checked a relatively narrow range of transaction cases.

In order to ensure better verification, constrained random transactions generation was implemented using sequence library. Different data transactions and other combinations were included to visit previously unchecked states. Several sessions of simulating were performed keeping track of the coverage report.

Thus, as a final result, the coverage rate reached about 95.85%, which includes statement coverage and complete FSMs coverage. The rest of unvisited bins mostly includes rare

combinations that need more directed scenarios.

7.3 PROTOCOL COMPLIANCE IN DRIVER

Currently the driver implementation added another level of complication, as the transactions had to behave according to the DUT interface. The driver was supposed to translate the transaction-level request into clock-driven interface events in the correct order and timings

The initial difficulties included:

- Starting signal synchronization
- Completion of transaction management
- Keeping address constant
- Switching between read/write modes

Poor synchronization often caused incorrect transactions to be issued and observed by the monitor incorrectly.The solution came from reworking the driver process flow to rely on the interface clocking module with proper sequencing of interface signals.

Furthermore, additional tests were added to ensure the proper transaction protocol was followed during simulation.Ultimately, the implemented driver ensured the generation of valid transactions and communication with the DUT.

7.4 STATE MACHINE VERIFICATION COMPLEXITY

Verifying the bank state machine grew harder with increasing complexity of the transactions. A number of time-dependent states were part of the FSM:

IDLE → ACT_CMD → tRCD → COL_CMD → DATA → PRE → DONE

Given tht the transitions between states were time and operation type dependent, it would be too complex to validate each one individually. The following aspects needed special focus for

validation:

- Proper conditions for entering a state
- Completion of transitions
- Prolongation of states
- Handling recovery

Functional coverage and assertions were added to trace the FSM.The waveforms indicated that each state and transition of the FSM had been successfully traversed. As a result, full FSM state and transition coverage was obtained.

CHAPTER 8

FUTURE ENHANCEMENTS

8.1 INTRODUCTION

The While the suggested design has been successful at implementing the verification of the memory controller architecture inspired by the HBM4 standard, the architecture itself is a relatively simple one used for experimental purposes. The evolution of memory architectures in terms of increasing bandwidth and power efficiency and using more elaborate verification methods continues. Thus, there are certain improvements that can be made to the system in the future.

The potential improvements mentioned in this chapter are aimed at widening protocol support, improving the completeness of the verification process, applying new verification approaches, and increasing the system capability.

8.2 FULL HBM 4 SPECIFICATION COMPLAINCE

Productivity In the current design implementation, some concepts have been incorporated that have been borrowed from the HBM4 architecture and its verification process. Nonetheless, in industry HBM4 chips, there is far more protocol complexity, bigger memory organizations and stricter timings specified in JEDEC specifications.The next big improvement to be made should be that of expanding the implemented memory organization to incorporate full support for HBM4 protocols.

Some of the elements that could be included in the design implementation in the future are:

- Several memory channels
- More command encodings
- Refreshing commands
- Bursting operations
- Bank management capabilities
- Address map options
- Dependency between timings parameters

Currently, in the implemented architecture, transactions occur on a functional level only. The full protocol would require several protocol-related improvements including command arbitration and scheduling and capability for concurrent memory accesses.Another improvement to be made would involve increasing the memory size and using hierarchical memory organizations, as seen in actual HBM memories.

This would improve the simulation reality and allow testing of more complicated access patterns. Also, another improvement could be incorporation of JEDEC timing requirements into the implementation to achieve realistic command latencies. This would make the entire project a real verification environment for industrial memory architectures.

8.3 FORMAL VERIFICATION INCLUSION

The verification methodology created in this project mainly uses the simulation technique along with directed testing, constrained random verification, assertions, and functional coverage techniques. While simulation is a very powerful tool that increases the confidence level in functionality, it cannot guarantee the behavior of the design in all possible input combinations. Using formal verification will help greatly increase the validity of the test by ensuring correctness using mathematical proofs.

- Some potential uses of formal verification that will be useful in the future include:
- Properties verification
- State Reachability
- Deadlocks
- Sequential Equivalence Checking
- Protocol Correctness

As opposed to simulations, formal verification tests all the possible states in all possible state combinations under the set constraints and automatically finds out any violations in the

process.The importance of formal verification becomes more evident in the case of memory architecture designs like HBM4 because it is much harder to verify their commands and states.

- Some possible formal properties include:
- Reading should give the value previously written
- Illegal state transitions should never happen
- Disabling DQ outside the data windows
- All transactions should have an end point

8.4 MULTISTACK VERIFICATION

Current implementation consists of a single memory bank designed based on the HBM4 architecture. Commercial HBM solutions usually comprise several stacks of memory connected via common interfaces and control mechanisms. Verification could be further enhanced to support multi-stack memory architectures in the future.

Possible improvements include:

- Multiple memory bank instances
- Transaction parallelism
- Scheduling across banks
- Arbitration algorithms
- Common controllers

Multi-stack memory verification raises new problems related to synchronization, load distribution, and arbitration.

Some possible future verification tasks are listed below:

- Simultaneous read and write operations
- Collisions processing
- Bandwidth management
- Burst scheduling concurrency

This enhancement will enable realistic modeling of multi-stack memory behavior under high performance demands.One other interesting improvement would be the inclusion of traffic generation algorithms that mimic real-world use cases like AI accelerators or data centers.

Such verification will help gain insight into system scaling. As seen from the improvements proposed in this chapter, it can be stated that the currently designed system provides a platform from which further advancements in research in memory validation can be achieved. With an extension to protocol validation and the inclusion of formal validation, along with power-aware and multi-stack approaches, a scalable and industry-oriented validation mechanism can be designed.

CHAPTER 9
CONCLUSION

9.1 INTRODUCTION

The The main focus of this chapter is on summarizing the entire effort done throughout the project and presenting its complete findings as far as results obtained from implementation and verification are concerned. This chapter focuses on the important contributions made by the proposed HBM4 memory architecture, its results, and also talks about the constraints involved and concluding the effort. The aim of this project was not just to implement memory functions but also to develop an effective verification methodology for protocol verification and measuring the extent of verification.

9.2 SUMMARY OF THE WORK DONE

This paper discussed the design and verification of HBM4-inspired memory architecture using SystemVerilog and UVM methodologies. The aim of the project was to develop a functional RTL realization which can perform memory operations meeting timing requirements, and verify its functionality through simulation. The first part of this project involved the comprehension of the historical context, as well as motivation of high-bandwidth memory architectures. In accordance with these concepts, the development of a simplified bank-level memory model with programmable timings and DDR-like data transfers took place. A number of architectural blocks such as command execution, timing controls, address handling, memory storing, bursting and interfacing have been added to the RTL realization of the HBM memory.

Finally, the complete UVM verification environment was developed containing transaction objects, sequence libraries, drivers, monitors, scoreboards, coverage collecters, and test environment.

The verification process has been divided into three stages as follows:

- RTL verification – using Quartus
- Early UVM development – using EDA Playground
- Verification – using QuestaSim

At the first stage, directed tests were used to validate functionality for known cases, while the second verification phase utilized randomization to cover the edge cases. In addition to verification, SVA and coverage-driven techniques were used for protocol checking.

9.3 OUTCOMES

The proposed solution fully addressed the key goals set out at the start of the project, successfully proving the whole flow from design to verification of a memory system based on HBM4.One of the key successes that can be attributed to this project was the implementation of a parameterizable RTL memory controller capable of performing read and write transactions with controllable timing behavior.

The designed FSM accurately completed memory commands and ensured their proper sequencing and synchronization both with internal processes and external bus activity.In terms of verification, a reusable UVM environment, which could perform both directed and constrained random test scenarios, was successfully created.

The proposed environment effectively performed:

- Transaction-level verification
- Scoreboard automatic checking
- Collection of functional coverage
- Assertions monitoring
- Debugging with waveforms

Metric	Result
Overall Coverage	**95.85%**
Statement Coverage	**100.00%**
Branch Coverage	**98.46%**
Toggle Coverage	**98.50%**
FSM Coverage	**100.00%**
Assertion Coverage	**71.42%**
UVM Errors	**0**
UVM Fatal	**0**

9.4 LIMITATION

Despite the success in implementation and verification, there are still some limitations because of practical scope restrictions.The HBM4 architecture that has been employed in this project is only an example of how HBM4 works and cannot be considered full HBM4 industrial implementation. Some simplifications have been applied on purpose in order to keep development manageable.

Current limitations of this implementation:

- Use of only selected memory operations rather than full JEDEC commands.
- Use of simplified memory organization.
- Absence of full hierarchy of channels.
- Absence of physical design flow.
- Absence of post-layout verification.

- Absence of full approach to timing closure.
- Verification through simulation rather than proof.

Finally, the lower coverage in terms of assertions compared to functional coverage means room for improving protocol checking. Also, the implementation has been tested only via simulations and no FPGA or silicon testing has been done.Such limitations do not affect the credibility of the implementation but mark certain boundaries for further expansion of its functionality.

REFERENCES

[1] JEDEC Solid State Technology Association, *High Bandwidth Memory (HBM4) DRAM Standard*, JESD270-4, latest available revision.

[2] JEDEC Solid State Technology Association, *High Bandwidth Memory (HBM3) DRAM Standard*, JESD238.

[3] S. Agarwal et al., *Resistive Memory Device Requirements for a Neural Algorithm Accelerator*, IEEE Transactions.

[4] J. Bergeron, *Writing Testbenches Using SystemVerilog*, Springer.

[5] C. Spear and G. Tumbush, *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*.

[6] Mentor Graphics, *QuestaSim User Manual*.

[7] Intel Corporation, *Quartus Prime User Guide*.

[8] Accellera Systems Initiative, *Universal Verification Methodology (UVM) 1.2 Reference Manual*.

[9] S. Sutherland, *SystemVerilog Assertions Handbook*.

[10] D. Harris and S. Harris, *Digital Design and Computer Architecture*.

[11] IEEE Standard 1800™, *SystemVerilog Language Reference Manual*.

[12] M. Keating et al., *Low Power Methodology Manual*, Springer.

[13] Cadence Design Systems, *Functional Verification Methodology Guide*.

[14] Mentor Graphics, *Coverage Driven Verification User Guide*.

[15] Recent HBM memory architecture and verification journal papers (2022–2025).